
SkillBridge: Harnessed Skill Transfer for Multi-Hop Visual Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Agentic skills can distill task-specific experience into reusable procedural knowl-
2 edge, enabling fast adaptation. Yet most skill-learning systems remain domain-
3 bound: skills learned in one environment are tied to its observations, action space,
4 and execution interface, making transfer difficult. Cross-domain skill transfer
5 therefore requires separating reusable procedural structure from domain-specific
6 grounding, inputs, preconditions, actions, and success criteria, especially across
7 heterogeneous tasks with large distribution shifts. To address this challenge, we
8 introduce SkillBridge, a harnessed skill-transfer framework for multi-hop visual
9 agents. SkillBridge represents skills as uniform, evidence-grounded protocols
10 rather than opaque, domain-isolated action plans. A runtime harness binds each
11 skill to domain-specific adapters, checks feasibility, traces execution, and vali-
12 dates outcomes, while an orchestrator coordinates the full lifecycle from rollout
13 collection and skill extraction and management. Skills are first discovered and
14 stress-tested in games, where dense rewards and deterministic resets provide in-
15 expensive verifiable multi-hop structure, and are then transferred to short-video
16 reasoning, web-agent, OS-agent, and visual-reasoning settings through few-shot
17 adapters. The actor agent converts visual observations into a unified schema,
18 retrieves applicable skills, and executes actions. The skill-bank agent extracts
19 skills from multi-hop reasoning traces and multi-step interactions, while the skill
20 crafter agent reflects on failures and revises skill protocols. By coupling visual
21 grounding, action execution, skill-bank retrieval, skill crafting, and orchestration
22 under a gate-bound harness, SkillBridge turns game-derived reasoning routines into
23 reusable protocols for evidence-grounded multi-hop VLM agents. Experiments
24 show that SkillBridge achieves a XX% improvement over baselines across all task
25 categories.

1 Introduction

27 Agents are increasingly deployed in interactive settings that require long-horizon decision-making
28 through iterative reasoning and tool use [1, 2]. This agentic paradigm has been widely adopted
29 across embodied AI [3], web and computer-use agents [4, 5], software engineering [6], and game-
30 playing [7, 8]. Unlike static question answering, these tasks require agents to observe the current state,
31 act through tools or environment interfaces, monitor feedback, and revise their plans over multiple
32 steps. However, as tasks grow more complex, unguided self-exploration becomes increasingly
33 ineffective: agents must discover recurring subgoals, compose multi-step behaviors, and recover from
34 failures under sparse feedback, while manual failure analysis, task decomposition, and reward design
35 quickly become impractical [9, 10]. **Skills** offer a natural abstraction for such behavior: they encode
36 reusable procedures for recurring subgoals, such as finding evidence, comparing objects, navigating

37 to a target, satisfying a constraint, or recovering from failure. By retaining and composing these
38 procedures, agents can reduce repeated exploration and improve long-horizon consistency [11, 12].

39 However, skills are difficult to acquire and reuse: useful skills often require expert knowledge
40 to specify feasible procedures and verification protocols [13, 14]. These procedures are typically
41 grounded in domain-specific observations, actions, tools, and constraints, and therefore cannot be
42 directly deployed across domains; for example, skills learned by a game-specific agent may not
43 transfer reliably to visual reasoning tasks [12]. Meanwhile, a single generalist policy is hard to train
44 across heterogeneous domains, where mismatched observations, action spaces, and reward signals
45 can induce interference and negative transfer [15].

46 Since acquiring skills from scratch in each new environment is costly, a natural alternative is to
47 transfer skills across domains. Although domains may differ in interfaces, dynamics, and feedback
48 signals, many agent tasks share a common *observe-act-feedback loop* [10, 16]. This suggests that
49 skills can be treated as adaptable protocols: their high-level intent and decision logic can be preserved,
50 while their grounding, tools, and success conditions are rebound to the target domain [17]. For
51 example, a *game agent* may learn to inspect a scene, identify a blocked path, search for a key object,
52 and verify that the path becomes reachable; a *multi-hop visual reasoning* task may follow the same
53 pattern by inspecting an image, locating relevant evidence, comparing it with context, and verifying
54 the answer. This raises our central question: *Can agents learn reusable skill protocols from interactive
55 experience and transfer them to new domains through lightweight adaptation?*

56 To address this gap, we introduce a **Harness**, *i.e.* an execution and validation layer that separates
57 the reusable procedural core of a skill from domain-specific grounding. By binding abstract skills
58 to domain-specific entities, preconditions, and admissible actions, the harness allows skills learned
59 in one environment to support decision-making in another rather than being rediscovered from
60 scratch [18, 19]. Prior work studies this separation for in-domain skill reuse [20, 21]; we extend it to
61 interactive cross-domain skill transfer. Among interactive domains, games offer a controlled sandbox
62 for pre-training skill discovery, learning reusable procedures, and calibrating agentic modules. Their
63 structured observations, explicit actions, configurable rewards, and accessible demonstrations make it
64 easier to mine skills from trajectories and adapt them through the harness. We therefore develop and
65 validate skill discovery and transfer in games, then extend the same pipeline to web, computer-use,
66 and multi-hop visual reasoning tasks, where sampling is costlier and feedback is sparse.

67 In this paper, we propose SkillBridge, an online skill-transfer framework for multimodal agents,
68 enabled by a harness with offline skill discovery, refinement, and promotion. SkillBridge consists of
69 three components: (i) a **Decision Agent** grounds visual observations from games, computer-use tasks,
70 images, or videos into structured schemas, retrieves relevant skills, and executes actions. (ii) a **Skill
71 Bank Agent** discover reusable skills from Decision Agent rollouts, refines existing candidates or
72 crafts new skills when discovered skills do not meet task demands via reflection, and proposes them
73 for promotion. (iii) a **Harness** validates candidate skills and adapts their domain-agnostic procedural
74 structure to domain-specific entities, preconditions, and admissible actions. The three components
75 interact in both online and offline phases. Online, the Decision Agent queries the Harness with the
76 current state and domain context; the Harness retrieves and grounds feasible skills for execution.
77 Offline, the Skill Bank Agent mines, refines, crafts, and promotes skills from rollouts, while the
78 Harness validates their cross-domain feasibility and generalizability before they enter the skill bank.
79 We use a mixed-model training strategy. The **Decision Agent** and **Skill Bank Agent** are optimized
80 with Group Relative Policy Optimization (GRPO) [22] for decision-making and skill-management
81 behaviors. For reasoning-intensive components, such as skill crafting and harness grounding, we use
82 a stronger teacher model to generate high-quality supervision and validation signals. To the best of
83 our knowledge, SkillBridge is the first harness-based framework for skill transfer in visual agents. By
84 decoupling reusable skill procedures from domain-specific grounding, SkillBridge enables continual
85 refinement and few-shot adaptation across interactive control and multi-hop visual reasoning tasks.
86 Our contributions include:

- 87 • We introduce a harness-based skill transfer framework for visual agents, where the harness
88 performs online skill adaptation by grounding reusable procedures to the current domain
89 through entity binding, precondition checking, and admissible action filtering.
- 90 • We develop a skill discovery and crafting pipeline that mines reusable skills from decision
91 agent rollouts, refines incomplete candidates through reflection, and promotes validated
92 skills into a continually improving skill bank.

- We conduct experiments across interactive decisioning in both computer use and video games, and multi-hop visual reasoning tasks, showing that SkillBridge improves skill reuse, supports few-shot cross-domain adaptation, and achieves strong performance by achieving XX% better performance compared with non-transfer and non-harnessed baselines.

2 Related Work

2.1 Self-Improving Vision Language Models and Agents

Self-improvement has become a common strategy for improving multimodal models and agents without dense human supervision. Vision-Zero [23] casts visual reasoning as gamified self-play over arbitrary image pairs, Vision-SR1 [24] uses self-generated perceptual outputs as reward signals, and VisPlay [25] trains a questioner–reasoner on unlabeled images with GRPO and difficulty-aware rewards. Recent work extends this paradigm to interactive agents: Active-Zero [26] co-evolves Searcher, Questioner, and Solver agents, while V-Zero [27] uses Questioner–Solver co-evolution with pseudo-label refinement. Related mechanisms appear in VLM agents, including SE-GUI [28] for GUI grounding, GUIShift [29] for offline affordance learning, and EvoVLA [30] for long-horizon embodied control. Agent-centric methods also use experience for improvement: EvolveR [31] distills trajectories into strategic principles, ERL [32] converts successes and failures into retrievable heuristics, and SEA [33] learns from verifiable GUI trajectories. While these methods reduce supervision costs, they usually optimize behavior within the same task family, interface, or agent loop that produced the training signal, making the learned improvement hard to inspect, compose, or transfer. Our work instead extracts multi-hop reasoning procedures as reusable VLM-agent skills, refines them with evidence and failure feedback, and reuses them across heterogeneous tasks.

2.2 Reusable Agent Skills and Cross-Task Transfer

Recent work studies how agents convert interaction traces into reusable skills instead of solving each task from scratch. PolySkill [34] separates abstract skill goals from site-specific implementations, improving generalization across websites and tools. Trace2Skill [35] distills trajectory-local lessons into compact skills for future tasks, while XSkill [36] extracts task-level workflows and action-level experiences from multimodal rollouts, grounding retrieval in the current visual observation. Other methods focus on continually updating skill or memory structures. MemRL [37] learns episodic-memory retrieval utilities for frozen LLM agents. MemSkill [38] treats memory construction as a reusable skill refined through a controller–executor–designer loop. UI-Mem [39] stores workflows, subskills, and failure patterns for GUI agents. SkillRL [14] distills trajectories into a hierarchical SkillBank and co-evolves it with the policy. EvoSkill [40] mines failure-derived skills and keeps those that improve held-out validation. Skill transfer has also been studied beyond LLM agents. XIRL [41] learns cross-embodiment visual representations for imitation and inverse RL. Closely related to our setting, COS-PLAY [12] couples a decision agent with a skill-bank agent, allowing long-horizon game agents to improve both skill use and skill discovery from unlabeled rollouts. These works support cross-task transfer, but usually within bounded environments, tools, memory operations, or agent modalities. Our work instead targets heterogeneous transfer across games, web agents, OS agents, video understanding, and visual reasoning through a unified harness for skill instantiation, grounding, execution, and validation.

2.3 Agent Knowledge Generation and Runtime Harnesses

Agent knowledge generation and runtime harnesses are central to agents that improve from experience, reuse procedural knowledge, and execute workflows beyond a single prompt. Agent Workflow Memory [42] induces reusable workflows from web-navigation traces, while AFlow [43] optimizes code-represented workflows with execution feedback. Procedural-memory methods offer non-parametric adaptation: ProcMEM [44] converts episodic traces into executable memories with activation, execution, and termination conditions; Memento [45] improves agents through stored experience and retrieval; and hierarchical procedural knowledge can improve agentic LLM workflows, sometimes allowing smaller models to outperform larger baselines [46]. ReMe [47] and Mem^P [48] further argue that procedural memory should be refined and updated rather than kept as a static append-only store. Complementary work studies the agent harness: the runtime layer managing context, artifacts, tools, execution state, and coordination. Recent harness studies show that persistent artifacts,

structured handoff, and planner–executor–evaluator decomposition improve long-running agent reliability [18, 19]. Natural-Language Agent Harnesses [49] proposes editable harness specifications executed by a runtime with contracts, durable artifacts, and adapters. AutoHarness [50] synthesizes code harnesses that enforce valid behavior, while Meta-Harness [51] optimizes harness code for storage, retrieval, and model inputs. Together, these works suggest that reusable knowledge needs an execution layer: for cross-domain skill transfer, a harness must instantiate skills, bind them to observations, manage context, validate outcomes, and adapt workflows across multi-disciplinary tasks.

3 Problem Setting

Preliminary. We model each task as an interactive decision process $\mathcal{E} = (\mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, T)$, where $\mathcal{O}$ is the observation space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the transition dynamics, $\mathcal{R}$ is a task reward or evaluation signal, γ is the discount factor, and T is the horizon. This formulation covers both interactive control and multimodal reasoning tasks. The observation o_t may be a game state, web page, desktop screen, image, or video context, optionally paired with a task instruction or question. The action a_t is drawn from a domain-specific interface, including primitive environment actions, tool invocations, reasoning operations, skill transitions, and terminal prediction or commit actions.

At timestep t , the agent observes o_t , maintains a short-horizon intention z_t , uses an active skill s_t , selects an action a_t , receives feedback r_t , and transitions to o_{t+1} . A rollout is represented as a sequence of transition tuples

$$\tau = (e_1, \dots, e_T), \quad e_t = (o_t, z_t, s_t, a_t, r_t, o_{t+1}, d_t), \quad (1)$$

where d_t indicates whether the episode terminates after step t . The intention z_t captures the agent’s local objective or strategic interpretation, while the skill s_t provides reusable procedural guidance over one or more steps. Both may be revised along the rollout as the agent gathers evidence, interacts with the environment, and updates its decision strategy [14, 12].

Transferable Skill Abstractions. We define a skill as a reusable procedure grounded in the target domain. Each skill s is represented as

$$s = (g, \mathcal{Z}, p, \rho, c, q), \quad (2)$$

where g is the skill goal, $\mathcal{Z}$ are typed slots, p specifies preconditions, ρ is the procedural routine, c defines completion criteria, and q is an evidence or effect contract. Typed slots form the transfer interface: the same routine can bind to game entities, web elements, desktop regions, image objects, or video segments. Thus, a skill is not a fixed action sequence, but a reusable pattern for perception, evidence gathering, reasoning, and action selection.

Let $\mathcal{B}$ denote the current skill bank. During execution, the active skill $s_t \in \mathcal{B}$ guides the policy through its goal, routine, and contract. Transfer is achieved by preserving ρ and q , while rebinding $\mathcal{Z}$ through the target-domain harness.

Harness for Skill-Guided Decision-Making. The harness is the runtime interface between abstract skills and domain-specific execution. Given an observation o_t , intention z_t , and active skill s_t , it grounds typed slots, checks preconditions, exposes valid actions or evidence operations, and records execution traces. The policy acts through this grounded skill context:

$$a_t \sim \pi_\theta(\cdot \mid o_t, z_t, H(o_t, z_t, s_t)), \quad (3)$$

where $H(\cdot)$ denotes the grounded execution context produced by the harness. This context binds the abstract skill to the current domain, restricts the policy to valid operations, and records whether the skill is progressing as expected. The same mechanism is used before promotion: a candidate skill must execute consistently under the harness, rather than merely explain a particular trajectory. In this way, the harness keeps skill execution grounded and traceable without tying the skill itself to one domain.

Skill Extraction and Crafting. Skills are obtained from agent rollouts through a gated lifecycle. At update iteration $u + 1$, let $\mathcal{D}^{(u+1)} = \{\tau_i\}$ denote the newly collected rollouts. The extraction module decomposes these rollouts into candidate segments and converts them into extracted draft skills:

$$\hat{\mathcal{S}}^{(u+1)} = \phi_{\text{ext}}(\mathcal{D}^{(u+1)}) = \{\hat{s}_j\}_{j=1}^M. \quad (4)$$

Each extracted skill $\hat{s}_j$ contains an inferred intent, typed slots, preconditions, routine, completion criteria, and evidence contract. These extracted skills can already serve as candidates when their structure is sufficiently general. For skills that remain overly trajectory-specific, a skill crafter further generalizes or repairs them by abstracting away domain-specific predicates and rewriting them as typed protocols over shared reasoning and action operators:

$$\tilde{s}_j = \phi_{\text{craft}}(\hat{s}_j). \quad (5)$$

This produces a combined candidate set containing both directly extracted skills and crafted variants:

$$\mathcal{C}^{(u+1)} = \hat{\mathcal{S}}^{(u+1)} \cup \{\tilde{s}_j\}_{j=1}^M. \quad (6)$$

The skills in $\mathcal{C}^{(u+1)}$ are candidates for promotion, but are not added to the active bank until they pass harness-mediated validation.

Skill Bank Maintenance. Validated skills are stored in a refineable skill bank $\mathcal{B}$. At update iteration $u + 1$, the bank is revised using the previous bank and the candidate skills proposed by the extraction–crafting pipeline:

$$\mathcal{B}^{(u+1)} = \Phi_{\text{bank}}(\mathcal{B}^{(u)}, \mathcal{C}^{(u+1)}) \quad (7)$$

The operator Φ_{bank} summarizes the bank maintenance actions. Before a crafted skill is activated, the harness applies a lightweight validation check, encouraging the bank to retain reusable procedures rather than trajectory-specific shortcuts.

4 Methodology

We propose a skill-based multi-agent VLM framework for transferable multi-hop visual reasoning, organized around three agents. The **Decision Agent** serves as the online policy: it consumes structured schemas from visual grounding and tool use, queries the harness for feasible skills, and executes multi-step actions. The **Harness Agent** grounds retrieved skills to the current task and domain by binding entities, checking preconditions and evidence requirements, and filtering admissible actions. The **Skill Bank Agent** discovers reusable skills from rollouts, refines them through failure reflection, crafts new skills for unmet task demands, and manages promotion, rollback, and bank maintenance. We instantiate this system with a student–teacher design, where smaller trainable models handle online decision-making and skill use, while larger teacher models provide skill discovery, failure diagnosis, refinement, and validation.

4.1 Schema-Based Decision Agent

The decision agent is the online controller that converts structured observations and retrieved skills into executable actions. At each timestep, it first grounds the visual state into a structured schema and updates a short-horizon intention that captures its current subgoal. This intention determines whether the agent should continue the active skill or query the skill bank for a new one. When switching is needed, the agent retrieves candidate skills and uses the harness to adapt them to the current domain by binding task-specific entities, checking preconditions, and exposing admissible actions. The grounded skill then serves as procedural context for action selection, while the agent produces the final primitive action through the domain interface.

Visual Schema Grounding. We ground visual observations into structured schemas through a tool-calling protocol. Given a screenshot, image, or video frame, the model queries visual and reasoning tools to identify entities, relations, state attributes, admissible actions, and supporting evidence. These tools map domain-specific visual content, *e.g.*, UI elements in web and OS environments, objects in images, or characters and events in videos, into typed textual representations. Additional reasoning tools compute properties such as counts, distances, and comparisons, while logging intermediate derivations for evidence tracing. The collected tool outputs are then composed into a structured, machine-readable schema used by downstream multi-hop reasoning and skill grounding. We provide details of the schema-construction tools and schema examples in Appendix B.

Harness-based skill query. At runtime, the decision agent maintains an active skill and a current intention. The intention is represented by a short textual summary and a discrete tag that describe the

current subgoal, and is updated after each observation. A stable intention indicates that the agent should continue the active skill, while a sharp intention shift marks a subtask transition and triggers skill switching [52]. When switching is triggered, the decision agent retrieves candidate skills from the skill bank using Retrieval-Augmented Generation (RAG) [53]. In our framework, skills are textual procedural plans that guide action selection rather than executable policies. The harness grounds each candidate in the current domain by binding entities, checking preconditions, and filtering admissible actions. The decision agent then selects among the grounded candidates and executes the selected skill.

Action execution. After skill grounding and selection, the decision agent produces a primitive action through the domain harness, conditioned on the current schema, task instruction, recent history, intention, and grounded active skill. The skill serves as a procedural reference rather than a closed-loop controller: it suggests the next substep and maintains multi-step coherence, while the agent still chooses the concrete action. The harness filters inadmissible actions and maps the selected action to an environment command, tool call, or reasoning operation. The resulting observation and feedback are recorded in the trace and used to update the schema, intention, and active-skill status.

4.2 Cross-Domain Skill Harness

A central difficulty in cross-domain skill transfer is that a skill learned in one environment is rarely executable as-is in another. Its predicates, entity types, action interface, and evidence requirements may all be coupled to the source domain. At the same time, these domains share a common interaction structure: the agent observes an external state, reasons over a multi-step objective, invokes environment actions or tools, and updates its decision process from feedback. The main variation lies in the domain interface: what observations are exposed, which actions are admissible, what tools are available, and how evidence is represented. We therefore introduce a *cross-domain skill harness* that factors this domain-specific interface out of the skill itself. The harness treats each domain as a switchable bundle of observation schemas, action protocols, tool APIs, and grounding rules, while preserving the reusable procedural content of a skill. It plays two complementary roles: (i) an *online runtime role* that grounds, adapts, and filters skills before actor execution, and (ii) an *offline validation role* that tests whether skills are reliable enough to be promoted to, or retained in, the skill bank.

Cross-domain harness. The harness is the online runtime interface that makes retrieved skills usable outside the domain in which they were learned. At each actor step, it takes the current schema state and a set of retrieved skills, and narrows the candidates through lightweight admissibility checks: skill status, feasible domain and task, availability of a target-domain adapter, and adapter-level applicability. For each admissible candidate, the harness translates source-domain predicates into the target predicate vocabulary, binds typed slots in the skill protocol to entities or evidence spans in the current schema, and checks whether the proposed invocation satisfies required bindings, preconditions, and evidence constraints. The actor still chooses among the grounded candidates and emits the next primitive action. Thus, the harness does not replace the policy or score skills; it only exposes valid invocations, vetoes invalid ones, and routes the selected operation through the corresponding domain adapter. This separates the reusable procedural content of a skill from domain-specific grounding, action interfaces, and evidence conventions.

Skill validation. The same harness also serves as an offline verifier for skill-bank updates. For each candidate skill, it records structured skill episodes, checks evidence-grounded execution, replays prior successful traces, evaluates few-shot transfer to target domains, and reports pass/fail diagnostics to the promotion pipeline. Skills that fail validation are not silently discarded: their failure modes, such as missing bindings, predicate-vocabulary mismatch, failed preconditions, or low cross-domain admit rate, are routed back to the crafter for repair, generalization, or retirement. In this way, the harness acts as an online admissibility layer during inference and as an offline gate for maintaining a transferable skill bank.

4.3 Skill Discovery, Crafting, and Promotion

The skill-bank agent is the offline component that turns decision-agent rollouts into reusable procedural knowledge. Given interaction traces, it identifies recurring subgoals, groups them into skill candidates, estimates their preconditions and effects, and maintains a versioned bank of skills over time. Its update pipeline separates discovery, refinement, and promotion: rollout segments provide

289 candidate skills, failure traces trigger repairs or new drafts, and a gated lifecycle determines which
290 skills become active. This design lets the bank grow from experience while avoiding uncontrolled
291 accumulation, stale routines, or unverified cross-domain skills.

292 **Skill discovery.** Given decision-agent rollouts, the skill-bank agent extracts reusable segments and
293 matches them to the current bank. It first proposes candidate boundaries from local signals that
294 usually indicate a subtask change: predicate flips in the schema, changes in the intention tag, reward
295 or event spikes, surprisal peaks, and switches between primitive action execution and skill selection.
296 Nearby boundaries are merged to avoid over-segmentation. The agent then chooses a segmentation
297 over these candidates and assigns each segment either to an existing skill or to a new draft. A
298 segment is matched to an existing skill when its observed additive and deletive predicate changes
299 agree with the skill’s learned contract; otherwise, it is kept as a new skill candidate with provisional
300 preconditions, effects, and supporting traces. The resulting skill-candidate bundles are forwarded to
301 refinement and crafting.

302 **Skill refinement and crafting via reflection.** Segmentation produces useful candidates, but it does
303 not resolve all failure modes. A retrieved skill may fail because its slots cannot be bound in the
304 current state, its preconditions are too narrow, its description no longer matches the observed behavior,
305 its predicates are tied to the source domain, or the bank simply lacks the needed routine. We handle
306 these cases with a separate Skill Crafter. The crafter reads failure traces produced by the harness,
307 but does not edit the active bank. Each trace stores the attempted skill, the grounded slot values, the
308 failed step, the failure type, the local schema state, and the harness diagnostic. Repeated traces are
309 grouped into failure patterns keyed by the skill id, failure type, and failed step.

310 For each recurring pattern, the crafter proposes one of five draft mutations. (i) A *patch* edits an
311 existing skill when the intended routine is still useful but one step, precondition, or effect is wrong.
312 (ii) A *generalization* removes source-specific details, such as grid cells, UI ids, or frame indices,
313 and replaces them with typed slots that can be rebound in other domains. (iii) A *composition* joins
314 compatible skills when one skill’s effect satisfies another skill’s precondition. (iv) A *hypothesis*
315 creates a new draft skill for an uncovered subgoal that appears repeatedly in failures. (v) A *retirement*
316 proposal marks a skill as obsolete when it is subsumed by a more general skill or fails persistently
317 under valid invocations. All crafter outputs remain draft proposals, so reflection can repair and expand
318 the bank without directly changing which skills are executable.

319 **Skill promotion and bank management.** Candidate skills are admitted through an audited lifecycle
320 rather than added directly to the runtime bank. New skills first enter as DRAFT records produced by
321 discovery or crafting. After basic contract and provenance checks, they become CANDIDATE skills
322 for validation. Candidates may run in SHADOW mode, where they are evaluated without affecting the
323 actor’s decisions, or remain PROVISIONAL when they pass only part of the gate. A skill becomes
324 ACTIVE only after passing static checks, replay validation, shadow execution, few-shot transfer
325 tests, and non-regression tests. Skills that are superseded by more general candidates are moved to
326 DEPRECATED; candidates that fail validation are marked REJECTED. All transitions are made by a
327 single lifecycle manager, which records the evaluation result, content hash, verified domains, and
328 supporting traces before changing the skill state. This keeps discovery and crafting separate from
329 execution: they can propose or repair skills, but only the promotion gate decides which skills are
330 usable at runtime. ← point out teh lifecycle in the figure

331 4.4 Training and Skill Co-Evolution

332 We use a student–teacher setup because online interaction requires a fast trainable model, while
333 reliable skill discovery and validation benefit from stronger but more expensive supervision. The
334 student is the rollout-time model: a shared backbone with role-specific LoRA adapters for decision
335 making, skill selection, action execution, segmentation, contract learning, and curation. The teacher
336 is a larger frozen model used off-policy for cold-start labels, trajectory critique, skill repair, contract
337 checks, and judge-based scores. This keeps GRPO training lightweight while giving the skill bank
338 stable supervision during early construction, failure diagnosis, and promotion.

339 **Per-task co-evolution.** We use games as a controlled pre-training stage for learning skill use,
340 discovery, revision, and in-domain transfer. Starting from teacher-labeled cold-start rollouts, a shared
341 student backbone is trained in a closed loop. The decision agent interacts with each game using the
342 current skill bank, learns when to continue or switch skills, and executes primitive actions conditioned

on grounded skills. The skill-bank agent then analyzes these trajectories to segment reusable behavior, infer effect contracts, score candidate skills, and update the bank through refinement, merging, splitting, and retirement. In parallel, the crafter proposes repairs, compositions, generalizations, or new skills from repeated failures and high-value traces; the promotion pipeline admits them only after static checks, replay, shadow execution, transfer tests, and non-regression. We optimize the system with role-specific LoRA adapters for skill selection, action execution, segmentation, contract learning, and curation, which avoids collapsing these roles into one behavior. This stage teaches the decision agent to use skills as procedural context, teaches the skill-bank agent to extract compact skills from trajectory evidence, and trains the lifecycle that turns noisy proposals into verified bank entries. Since the same bank is reused across the game suite, skills discovered in one task can be retrieved, grounded by the harness, adapted to related states or games, and refined through later rollouts.

Cross-domain few-shot adaptation. After game pre-training, we adapt the skill bank to non-game domains using a small number of target episodes. Games provide reusable procedures, such as evidence inspection, comparison, delayed-state tracking, recovery from invalid actions, and multi-step planning, while the harness handles domain-specific observations and actions. Each skill is stored as a textual protocol with typed slots, preconditions, success or abort criteria, and effect contracts, rather than as a game-specific policy. In OSWorld, BrowserGym, image reasoning, or video understanding, the harness converts observations into schemas, translates source predicates, binds slots to target entities or evidence spans, checks preconditions, and filters invalid actions. The decision agent uses the grounded skill as procedural context, but still chooses the final primitive action, tool call, or reasoning step. Successful target executions add evidence to the skill record; repeated grounding or contract failures trigger repair, specialization, or rejection. A skill is promoted only after replay, evidence-grounded contract checks, source non-regression, and held-out target tests, which limits unsupported game-specific transfer.

5 Experimental Results

Benchmarks. We evaluate SkillBridge across heterogeneous interactive and visual reasoning benchmarks. For interactive decision-making, we use video-game environments, including LMGameBench [54], Orak [55], and Gym-V [56]; web-based interaction benchmarks, including BrowserGym [57, 58]; and desktop operating-system tasks from OSWorld [59]. These benchmarks test whether agents can adapt reusable skills across games, web interfaces, and OS-level workflows. For multi-hop visual reasoning, we evaluate settings where the agent takes an image or video as input and uses tools to manipulate visual context or gather evidence before answering. We use TIR-Bench [60] and VisualToolBench [61] for tool-based image reasoning, and Video-Holmes [62] and SIV-Bench [63] for evidence discovery and temporal/social reasoning in videos. Details are provided in Appendix A and B.

Training Setting. We train the actor agent and the skill bank agent in game environments, where the system learns decision-making, skill extraction, and skill bank maintenance, together with the shared skill schema and visual grounding interface. For adaptation to new tasks, we freeze the skill bank agent and the learned skill inventory, and train only the actor agent to ground, select, and apply existing skills in the new domain.

We use Qwen3.5-9B [64] as the backbone model and Qwen3.5-35B-A3B [64] as teacher model, using GPT-5.4 [65] as SFT models.

Baselines. We compare SkillBridge with strong frontier LLMs, including GPT-5.4 [65], GEMINI-3.1-PRO [66], CLAUDE-4.6-SONNET [67], and QWEN3-VL-235B [64].

Vanilla Qwen3.5-9B and Qwen3.5-35B-A3B [64]

5.1 In-dist Adaptation

All results are run by 16 episodes.

Model	Single-Player				
	2048 Reward $\uparrow$	Tetris Reward $\uparrow$	CandyCrush Reward $\uparrow$	Super Mario Bros Reward $\uparrow$	Avg. Reward $\uparrow$
GPT-5.4	1126.6 $\pm$ 150.2	458.2 $\pm$ 203.5	532.6 $\pm$ 24.8	752.0 $\pm$ 35.7	--
GEMINI-3.1-PRO	813.3 $\pm$ 143.6	372.7 $\pm$ 157.7	334.3 $\pm$ 59.4	436.8 $\pm$ 86.1	--
CLAUDE-4.6-SONNET	945.0 $\pm$ 134.5	444.2 $\pm$ 182.6	328.6 $\pm$ 23.8	399.5 $\pm$ 53.4	--
QWEN3-VL-235B	--	--	--	--	--
QWEN3.5-9B	1025.2 $\pm$ 248.0	38.1 $\pm$ 14.5	504.0 $\pm$ 38.5	472.8 $\pm$ 152.0	510.0 $\pm$ 112.5
QWEN3.5-35B-A3B	958.5 $\pm$ 188.0	65.1 $\pm$ 18.0	508.6 $\pm$ 33.5	499.5 $\pm$ 136.0	507.9 $\pm$ 99.5

Table 1: **Performance on the four high-signal env_wrappers games.** We report mean reward with **95% confidence intervals** for 2048, Tetris, Candy Crush, and Super Mario Bros. GPT-5.4 is evaluated with vision inputs over **20 episodes**, while Qwen3.5-9B and Qwen3.5-35B-A3B are evaluated with text inputs over **16 episodes**. Claude-4.6-Sonnet, Gemini-3.1-Pro, and Qwen3-VL-235B were not evaluated on these env_wrappers games in the provided sweep.

Model	Gym-V Single-Player								Avg.
	Air striker	Altered Beast	Columns	Dynamite Headdy	Space Harrier II	Streets of Rage 2	Strider	Thunder Force III	Reward ↑
GPT-5.4	52.5 ± 8.7	118.8 ± 43.8	153.6 ± 7.2	93.8 ± 18.8	23850.0 ± 7675.0	259.4 ± 61.2	0.0 ± 0.0	306.2 ± 193.8	---
GEMINI-3.1-Pro	97.5 ± 21.3	425.0 ± 218.8	99.2 ± 28.7	81.2 ± 18.8	22931.0 ± 7750.0	408.8 ± 84.3	112.5 ± 28.1	725.0 ± 187.5	---
CLAUDE-4.6-SONNET	93.8 ± 10.0	293.8 ± 18.8	62.9 ± 4.7	93.8 ± 18.8	29431.0 ± 11288.0	281.2 ± 55.6	31.2 ± 12.5	268.8 ± 93.7	---
QWEN3-VL-235B	67.5 ± 13.8	262.5 ± 31.3	131.6 ± 29.5	75.0 ± 18.8	14469.0 ± 56.0	201.9 ± 41.3	0.0 ± 0.0	750.0 ± 162.5	---

Table 2: **Performance on the Gym-V games with available records.** We report per-game mean reward with **95% bootstrap confidence intervals**, written as mean ± half-width of the interval. Bold indicates the best mean reward for each game. Games without records in the new result set are omitted.

Model	BrowserGym	OSWorld											Avg.
	MiniWoB++ Success Rate $\uparrow$	Chrome	GIMP	Calc	Impress	Writer	Multi Apps	OS	Thunder bird	VLC	VS Code	Overall Success Rate $\uparrow$	Success Rate $\uparrow$
GPT-5.4	59.2 $\pm$ 8.5	11.1	19.0	0.0	0.0	15.0	3.2	20.0	16.7	0.0	10.5	8.0 $\pm$ 3.5	33.6
GEMINI-3.1-PRO	65.6 $\pm$ 8.0	0.0	0.0	0.0	2.9	0.0	1.6	15.0	8.3	0.0	10.5	3.2 $\pm$ 2.0	34.4
CLAUDE-4.6-SONNET	65.6 $\pm$ 8.0	14.8	4.8	0.0	0.0	10.0	6.3	15.0	0.0	7.7	10.5	6.8 $\pm$ 3.5	36.2
QWEN3-VL-235B	58.4 $\pm$ 8.5	7.4	0.0	0.0	0.0	5.0	1.6	10.0	0.0	0.0	15.8	3.6 $\pm$ 2.5	31.0
QWEN3.5-9B	51.2 $\pm$ 8.5	7.4	0.0	0.0	0.0	0.0	0.0*	10.0	0.0	7.7	10.5	3.4 $\pm$ 2.5*	27.3
QWEN3.5-35B-A3B	36.0 $\pm$ 8.5	3.7	0.0	0.0	0.0	5.0	1.6	5.0	0.0	7.7	5.3	2.4 $\pm$ 2.0	19.2

Table 3: **Performance on BrowserGym MiniWoB++ and OSWorld.** We report **success rate** for BrowserGym MiniWoB++ and OSWorld. AssistantBench is excluded because all evaluated backbones receive zero reward under the provided judge. MiniWoB++ is evaluated over **125 episodes** per model. OSWorld is evaluated over **250 tasks** for most backbones. The final average is the unweighted mean of MiniWoB++ success rate and OSWorld overall success rate. *Qwen3.5-9B has a truncated OSWorld multi_apps shard, with 21 out of 63 tasks observed; its OSWorld overall denominator is therefore 208 rather than 250.

391 **5.3 Skills**

392 When the new skills being extracted / in what cases skills being crafted, how often teh skills being
393 promoted / rejected

394 Also work out a flow map for skills generated from games and other environment

395 **5.4 Multi-hop Visual Reasoning**

396 We evaluate a unified setting where the agent takes an image or video as input and uses tools
397 to manipulate the visual context or perform evidence-based reasoning before answering. TIR-
398 Bench [60] and VisualToolBench [61] test tool-based image reasoning, while Video-Holmes [62] and
399 SIV-Bench [63] evaluate evidence discovery and temporal/social reasoning in videos.

Model	Visual and Video Reasoning				
	VisualToolBench Success Rate $\uparrow$	TIR-Bench Success Rate $\uparrow$	SIV-Bench Success Rate $\uparrow$	Video-Holmes Success Rate $\uparrow$	Avg. Success Rate $\uparrow$
GPT-5.4	9.7 $\pm$ 3.5	34.0 $\pm$ 5.5	57.6 $\pm$ 4.5	39.6 $\pm$ 3.0	37.7
GEMINI-3.1-PRO	2.3 $\pm$ 2.0	18.7 $\pm$ 4.0	34.3 $\pm$ 4.5	6.4 $\pm$ 1.5	13.0
CLAUDE-4.6-SONNET	4.7 $\pm$ 2.5	27.0 $\pm$ 5.0	64.1 $\pm$ 5.0	45.8 $\pm$ 3.0	40.3
QWEN3-VL-235B	8.3 $\pm$ 3.0	21.7 $\pm$ 5.0	55.2 $\pm$ 5.0	35.7 $\pm$ 3.0	33.2
QWEN3.5-9B	6.0 $\pm$ 2.5	24.0 $\pm$ 4.5	53.4 $\pm$ 5.0	32.4 $\pm$ 2.5	31.2
QWEN3.5-35B-A3B	6.0 $\pm$ 2.5	24.3 $\pm$ 4.5	55.8 $\pm$ 5.0	37.9 $\pm$ 3.0	34.5

Table 4: **Performance on visual and video reasoning benchmarks.** We report **success rate** with **95% confidence intervals** on VisualToolBench, TIR-Bench, SIV-Bench, and Video-Holmes. The final average is computed as the pooled success rate over all benchmark examples, using $n = 300$ for VisualToolBench, $n = 300$ for TIR-Bench, $n = 382$ for SIV-Bench, and $n = 1000$ for Video-Holmes.

5.5 Ablation Study

SFT $\leftarrow$ SFT on game + SFT on all + SFT individual and GPRO individual

5.6 Discussion

6 Conclusion

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [2] Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165*, 2026.
- [3] Yi Wu, Zikang Xiong, Yiran Hu, Shreyash S Iyengar, Nan Jiang, Aniket Bera, Lin Tan, and Suresh Jagannathan. Selp: Generating safe and efficient task plans for robot agents with large language models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2599–2605. IEEE, 2025.
- [4] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.
- [5] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. *arXiv preprint arXiv:2410.05243*, 2024.
- [6] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [7] Weiyu Ma, Yuqian Fu, Zecheng Zhang, Guohao Li, and Bernard Ghanem. Vlms play starcraft ii: A benchmark and multimodal decision method. *arXiv preprint arXiv:2503.05383*, 2025.
- [8] Alex L Zhang, Thomas L Griffiths, Karthik R Narasimhan, and Ofir Press. Videogamebench: Can vision-language models complete popular video games? *arXiv preprint arXiv:2505.18134*, 2025.
- [9] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [11] Zizhao Wang, Jiaheng Hu, Caleb Chuck, Stephen Chen, Roberto Martín-Martín, Amy Zhang, Scott Niekum, and Peter Stone. Skild: Unsupervised skill discovery guided by factor interactions. *Advances in Neural Information Processing Systems*, 37:77748–77776, 2024.
- [12] Xiyang Wu, Zongxia Li, Guangyao Shi, Alexander Duffy, Tyler Marques, Matthew Lyle Olson, Tianyi Zhou, and Dinesh Manocha. Co-evolving llm decision and skill bank agents for long-horizon tasks, 2026. URL <https://arxiv.org/abs/2604.20987>.
- [13] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.
- [14] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.

- [15] Lirui Wang, Jialiang Zhao, Yilun Du, Edward H Adelson, and Russ Tedrake. Poco: Policy composition from and for heterogeneous robot learning. *arXiv preprint arXiv:2402.02511*, 2024.
- [16] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [17] Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026.
- [18] Justin Young. Effective harnesses for long-running agents. Anthropic, November 2025.
- [19] Prithvi Rajasekaran. Harness design for long-running application development. Anthropic, March 2026.
- [20] Gaodan Fang, Vatche Isahagian, KR Jayaram, Ritesh Kumar, Vinod Muthusamy, Punleuk Oum, and Gegi Thomas. Trajectory-informed memory generation for self-improving agent systems. *arXiv preprint arXiv:2603.10600*, 2026.
- [21] Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents. *arXiv preprint arXiv:2604.14004*, 2026.
- [22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [23] Qinsi Wang, Bo Liu, Tianyi Zhou, Jing Shi, Yueqian Lin, Yiran Chen, Hai Helen Li, Kun Wan, and Wentian Zhao. Vision-zero: Scalable vlm self-improvement via strategic gamified self-play. *arXiv preprint arXiv:2509.25541*, 2025.
- [24] Zongxia Li, Wenhao Yu, Chengsong Huang, Rui Liu, Zhenwen Liang, Fuxiao Liu, Jingxi Che, Dian Yu, Jordan Boyd-Graber, Haitao Mi, et al. Self-rewarding vision-language model via reasoning decomposition. *arXiv preprint arXiv:2508.19652*, 2025.
- [25] Yicheng He, Chengsong Huang, Zongxia Li, Jiabin Huang, and Yonghui Yang. Visplay: Self-evolving vision-language models from images. *arXiv preprint arXiv:2511.15661*, 2025.
- [26] Jinghan He, Junfeng Fang, Feng Xiong, Zijun Yao, Fei Shen, Haiyun Guo, Jinqiao Wang, and Tat-Seng Chua. Active zero: Self-evolving vision-language models through active environment exploration. *arXiv preprint arXiv:2602.11241*, 2026.
- [27] Han Wang, Yi Yang, Jingyuan Hu, Minfeng Zhu, and Wei Chen. V-zero: Self-improving multimodal reasoning with zero annotation. *arXiv preprint arXiv:2601.10094*, 2026.
- [28] Xinbin Yuan, Jian Zhang, Kaixin Li, Zhuoxuan Cai, Lujian Yao, Jie Chen, Enguang Wang, Qibin Hou, Jinwei Chen, Peng-Tao Jiang, et al. Se-gui: Enhancing visual grounding for gui agents via self-evolutionary reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [29] Longxi Gao, Li Zhang, Pengzhi Gao, Wei Liu, Jian Luan, and Mengwei Xu. Gui-shift: Enhancing vlm-based gui agents through self-supervised reinforcement learning. *arXiv preprint arXiv:2505.12493*, 2025.
- [30] Zeting Liu, Zida Yang, Zeyu Zhang, and Hao Tang. Evovla: Self-evolving vision-language-action model. *arXiv preprint arXiv:2511.16166*, 2025.
- [31] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xueming Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- [32] Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. Experiential reflective learning for self-improving llm agents. *arXiv preprint arXiv:2603.24639*, 2026.

- [33] Yuhao Cheng, Liang Tang, Shuxian Li, Yukang Huo, Tiaonan Duan, Kaer Huang, Yanzhe Jing, and Yiqiang Yan. Evolving in tasks: Empowering the multi-modality large language model as the computer use agent. *arXiv preprint arXiv:2508.04037*, 2025.
- [34] Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. Polyskill: Learning generalizable skills through polymorphic abstraction. *arXiv preprint arXiv:2510.15863*, 2025.
- [35] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjuan Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- [36] Guanyu Jiang, Zhaochen Su, Xiaoye Qu, et al. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026.
- [37] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026.
- [38] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.
- [39] Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. Ui-mem: Self-evolving experience memory for online reinforcement learning in mobile gui agents. *arXiv preprint arXiv:2602.05832*, 2026.
- [40] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- [41] Kevin Zakka, Andy Zeng, Pete Florence, Jonathan Tompson, Jeannette Bohg, and Debidatta Dwibedi. Xirl: Cross-embodiment inverse reinforcement learning. In *Conference on Robot Learning*, pages 537–546. PMLR, 2022.
- [42] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [43] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [44] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026.
- [45] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.
- [46] Vincent Hsiao, Mark Roberts, and Leslie Smith. Procedural knowledge improves agentic llm workflows. *arXiv preprint arXiv:2511.07568*, 2025.
- [47] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. *arXiv preprint arXiv:2512.10696*, 2025.
- [48] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.
- [49] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026.
- [50] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026.

- [51] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [52] George Konidaris, Scott Kuindersma, Roderic Grupen, and Andrew Barto. Constructing skill trees for reinforcement learning agents from demonstration trajectories. *Advances in neural information processing systems*, 23, 2010.
- [53] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [54] Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. Imggame-bench: How good are llms at playing games? *arXiv preprint arXiv:2505.15146*, 2025.
- [55] Dongmin Park, Minkyu Kim, Beongjun Choi, Junhyuck Kim, Keon Lee, Jonghyun Lee, Inkyu Park, Byeong-Uk Lee, Jaeyoung Hwang, Jaewoo Ahn, et al. Orak: A foundational benchmark for training and evaluating llm agents on diverse video games. *arXiv preprint arXiv:2506.03610*, 2025.
- [56] Fanqing Meng, Lingxiao Du, Jiawei Gu, Jiaqi Liao, Linjie Li, Zijian Wu, Xiangyan Liu, Ziqi Zhao, Mengkang Hu, Yue Zhang, et al. Gym-v: A unified vision environment system for agentic vision research. *arXiv e-prints*, pages arXiv-2603, 2026.
- [57] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=5298fKGmv3>. Expert Certification.
- [58] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/drouin24a.html>.
- [59] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- [60] Ming Li, Jike Zhong, Shitian Zhao, Haoquan Zhang, Shaoheng Lin, Yuxiang Lai, Chen Wei, Konstantinos Psounis, and Kaipeng Zhang. Tir-bench: A comprehensive benchmark for agentic thinking-with-images reasoning. *arXiv preprint arXiv:2511.01833*, 2025.
- [61] Xingang Guo, Utkarsh Tyagi, Advait Gosai, Paula Vergara, Jayeon Park, Ernesto Gabriel Hernández Montoya, Chen Bo Calvin Zhang, Bin Hu, Yunzhong He, Bing Liu, et al. Beyond seeing: Evaluating multimodal llms on tool-enabled image perception, transformation, and reasoning. *arXiv preprint arXiv:2510.12712*, 2025.
- [62] Junhao Cheng, Yuying Ge, Teng Wang, Yixiao Ge, Jing Liao, and Ying Shan. Video-holmes: Can mllm think like holmes for complex video reasoning? *arXiv preprint arXiv:2505.21374*, 2025.
- [63] Fanqi Kong, Weiqin Zu, Xinyu Chen, Yaodong Yang, Song-Chun Zhu, and Xue Feng. Siv-bench: A video benchmark for social interaction understanding and reasoning. *arXiv preprint arXiv:2506.05425*, 2025.

- 591 [64] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
592
- 593 [65] OpenAI. Introducing gpt-5.4, 2026.
- 594 [66] Google DeepMind. Gemini 3.1 pro model card, 2026.
- 595 [67] Anthropic. Introducing claude sonnet 4.6, 2026.

Domain	Benchmark	Total	Pool	Held-out	Modality
gymv (source)	13 retro envs	13 envs	~130 ep.	—	rendered frame
browser	VisualWebArena	910 tasks	200	50	screenshot + AXTree
	MiniWoB++	125 tasks	125	25	screenshot + AXTree
	AssistantBench	215 tasks	180	30	screenshot + AXTree
osworld	OSWorld	369 tasks	250	50	screenshot + AT-SPI tree
visual_reasoning	VisualToolBench	603 samples	300	100	image
	TIR-Bench	1,215 samples	300	100	image
video	Video-Holmes	1,837 q.	1,000	200	6 sampled frames
	SIV-Bench	8,728 q.	400	100	6 sampled frames
Total	—	—	2,635 ids + ~130 ep.	555 ids	—

Table 5: Benchmark sample counts and split rationale. “Total” is the upstream benchmark size. “Pool” is the stratified subset used for cold-start skill discovery, chosen to cover task diversity under annotation and rollout cost. “Held-out” is a disjoint subset reserved for E0/E1/E2 evaluation, chosen to provide stable comparisons under the compute budget.

A Benchmark Details

Table A summarizes the benchmark splits used across source and transfer domains. For each benchmark, we choose the *Pool* size to balance coverage and annotation/runtime cost: larger pools are assigned to heterogeneous benchmarks with broad task diversity, while smaller pools are used when tasks are more homogeneous or expensive to execute. The *held-out* split is sized to provide a stable evaluation estimate under the available compute budget.

B Toolset for Visual Reasoning

We construct visual schemas through a shared tool-calling layer that exposes domain-specific perception and state-query operators under a unified OpenAI-style function interface. All tools are registered through a common `ToolRegistry`, which defines reusable primitives such as `query_entity_pos`, `list_entities`, `check_relation`, `get_state_flags`, and `list_valid_actions`. Each domain wrapper binds these primitives to the current observation: `Gym-V` parses grid states and entities; `BrowserGym` queries the accessibility tree, bounding boxes, Set-of-Marks elements, and page metadata; `OSWorld` queries desktop UI elements and pixel locations from accessibility trees; and visual-reasoning wrappers provide image/video tools for object detection, region description, zooming, OCR, spatial queries, frame sampling, change detection, tracking, and temporal navigation. Reasoning tools further record intermediate derivations such as counts, comparisons, ratios, and claim verification. The Decision Agent uses these tool outputs to convert raw screenshots, frames, images, or videos into structured schemas containing entities, relations, state flags, valid actions, and evidence traces, which are then passed to the harness for skill grounding and execution.

C Visual Representation Schema

D Domain adapters

The harness implements each environment through a domain adapter, while keeping the skill representation fixed across domains:

- **Games.** The adapter maps abstract skill verbs to the game action vocabulary and evaluates contracts from pre/post state changes, such as reward increases, entity updates, and phase transitions.
- **Web and OS tasks.** The adapters ground slots to UI elements, accessibility nodes, or screen regions, restrict actions to available browser or desktop operations, and convert skill hops into clicks, typing, navigation, or system queries.

Field	Description
entities	Grounded visual elements with id, type, label, position, binding id, and ontology role.
attributes	Per-entity state or value, e.g., visible, focused, checked, disabled.
affordances	Abstract skill-level operators supported by each entity, e.g., inspect, select, open, compare.
relations	Spatial or semantic relations such as contains, adjacent, blocks, or grouped.
state_flags	Global task state, including progress, phase, scene type, errors, and dialog/input status.
targets	Task-relevant target, blocker, constraint, candidate set, and history anchor.
uncertainty	Field-level confidence annotations for uncertain entity attributes or roles.
actions	Plausible next actions, usually copied from the environment action space when available.
evidence	Reasoning-hop trace recording grounding, checking, retrieval, and conclusion steps.
evidence_refs	Addressable evidence records pointing to entities, frames, bounding boxes, DOM nodes, or tool outputs.
derivations	Typed symbolic reasoning records, such as counting, comparison, ratio, or verification operations.
answer	Final prediction with grounding entities, evidence chain, and confidence.

Table 6: Visual state schema used as the common interface between pixels, reasoning, skill retrieval, and action execution.

- **Image reasoning.** The adapter binds slots to objects, regions, attributes, or evidence spans, and routes hops to visual tools such as inspect, compare, count, and verify.
- **Video reasoning.** The adapter binds slots to frames, temporal segments, objects, events, or evidence spans, and routes hops to temporal operations such as sample, track, compare, verify, and answer.

Across domains, the actor sees the same schema, skill protocol, and admissible-action interface; only the grounding and execution backend changes.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- 786 • Providing as much information as possible in supplemental material (appended to the
787 paper) is recommended, but including URLs to data and code is permitted.

788 6. Experimental setting/details

789 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
790 rameters, how they were chosen, type of optimizer) necessary to understand the results?

791 Answer: **[TODO]**

792 Justification: **[TODO]**

793 Guidelines:

- 794 • The answer [N/A] means that the paper does not include experiments.
795 • The experimental setting should be presented in the core of the paper to a level of detail
796 that is necessary to appreciate the results and make sense of them.
797 • The full details can be provided either with the code, in appendix, or as supplemental
798 material.

799 7. Experiment statistical significance

800 Question: Does the paper report error bars suitably and correctly defined or other appropriate
801 information about the statistical significance of the experiments?

802 Answer: **[TODO]**

803 Justification: **[TODO]**

804 Guidelines:

- 805 • The answer [N/A] means that the paper does not include experiments.
806 • The authors should answer **[Yes]** if the results are accompanied by error bars, confidence
807 intervals, or statistical significance tests, at least for the experiments that support the
808 main claims of the paper.
809 • The factors of variability that the error bars are capturing should be clearly stated (for
810 example, train/test split, initialization, random drawing of some parameter, or overall
811 run with given experimental conditions).
812 • The method for calculating the error bars should be explained (closed form formula,
813 call to a library function, bootstrap, etc.)
814 • The assumptions made should be given (e.g., Normally distributed errors).
815 • It should be clear whether the error bar is the standard deviation or the standard error
816 of the mean.
817 • It is OK to report 1-sigma error bars, but one should state it. The authors should
818 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis
819 of Normality of errors is not verified.
820 • For asymmetric distributions, the authors should be careful not to show in tables or
821 figures symmetric error bars that would yield results that are out of range (e.g., negative
822 error rates).
823 • If error bars are reported in tables or plots, the authors should explain in the text how
824 they were calculated and reference the corresponding figures or tables in the text.

825 8. Experiments compute resources

826 Question: For each experiment, does the paper provide sufficient information on the com-
827 puter resources (type of compute workers, memory, time of execution) needed to reproduce
828 the experiments?

829 Answer: **[TODO]**

830 Justification: **[TODO]**

831 Guidelines:

- 832 • The answer [N/A] means that the paper does not include experiments.
833 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
834 or cloud provider, including relevant memory and storage.
835 • The paper should provide the amount of compute required for each of the individual
836 experimental runs as well as estimate the total compute.

- 837 • The paper should disclose whether the full research project required more compute
838 than the experiments reported in the paper (e.g., preliminary or failed experiments that
839 didn't make it into the paper).

840 **9. Code of ethics**

841 Question: Does the research conducted in the paper conform, in every respect, with the
842 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

843 Answer: **[TODO]**

844 Justification: **[TODO]**

845 Guidelines:

- 846 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
847 Ethics.
- 848 • If the authors answer [No], they should explain the special circumstances that require a
849 deviation from the Code of Ethics.
- 850 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
851 eration due to laws or regulations in their jurisdiction).

852 **10. Broader impacts**

853 Question: Does the paper discuss both potential positive societal impacts and negative
854 societal impacts of the work performed?

855 Answer: **[TODO]**

856 Justification: **[TODO]**

857 Guidelines:

- 858 • The answer [N/A] means that there is no societal impact of the work performed.
- 859 • If the authors answer [N/A] or [No], they should explain why their work has no societal
860 impact or why the paper does not address societal impact.
- 861 • Examples of negative societal impacts include potential malicious or unintended uses
862 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
863 (e.g., deployment of technologies that could make decisions that unfairly impact specific
864 groups), privacy considerations, and security considerations.
- 865 • The conference expects that many papers will be foundational research and not tied
866 to particular applications, let alone deployments. However, if there is a direct path to
867 any negative applications, the authors should point it out. For example, it is legitimate
868 to point out that an improvement in the quality of generative models could be used to
869 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
870 that a generic algorithm for optimizing neural networks could enable people to train
871 models that generate Deepfakes faster.
- 872 • The authors should consider possible harms that could arise when the technology is
873 being used as intended and functioning correctly, harms that could arise when the
874 technology is being used as intended but gives incorrect results, and harms following
875 from (intentional or unintentional) misuse of the technology.
- 876 • If there are negative societal impacts, the authors could also discuss possible mitigation
877 strategies (e.g., gated release of models, providing defenses in addition to attacks,
878 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
879 feedback over time, improving the efficiency and accessibility of ML).

880 **11. Safeguards**

881 Question: Does the paper describe safeguards that have been put in place for responsible
882 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
883 image generators, or scraped datasets)?

884 Answer: **[TODO]**

885 Justification: **[TODO]**

886 Guidelines:

- 887 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.